

Cloud Databases for Internet-of-Things Data

Phan Thi Anh Mai, Jukka K. Nurminen, Mario Di Francesco

Department of Computer Science and Engineering

Aalto University School of Science

Email: {mai.phan, jukka.k.nurminen, mario.di.francesco}@aalto.fi

Abstract—The Internet of Things (IoT) is posing new challenges and opportunities for data management and analysis techniques. One of the major problems is how to handle an increasing amount of data, with a variety of data types and data sources, in order to meet application-specific performance requirements. In this article, we address the suitability of different types of databases for storing and accessing IoT data in the cloud. Specifically, we compare the performance of SQL (i.e., MySQL) and NoSQL databases (i.e., MongoDB, CouchDB, and Redis) in a cloud environment by using different types of IoT data, namely, sensor readings and multimedia. We conducted a performance evaluation of the considered database systems through extensive experiments in the cloud. Our results show that SQL and NoSQL databases are both viable for IoT data, even though there is a high variability in the performance of the different NoSQL databases considered.

Keywords—cloud, Internet of Things, IoT, data, database, DBMS, RDBMS, NoSQL, SQL

I. INTRODUCTION

The development of pervasive computing, radio frequency identification (RFID) technologies and sensor networks has enabled the creation of the so-called *Internet of Things* (IoT) [1]. The major idea behind the IoT is that interconnected “smart” objects will form a global information and communication infrastructure for the creation of value-added services. The vision of the Internet of Things is represented by a huge, dynamic, and expandable network of networks, involving billions of entities such as sensors, actuators, RFID tags, computers, mobiles and so on. These devices simultaneously generate data and communicate with each other; as a consequence, a massive amount of data flows into the network. Besides, the types and the nature of the data are becoming more and more diverse. Data can be in several different formats, structured or unstructured, ranging from text and numbers to audio, pictures, and videos. Data are generated, stored, and transferred across multiple nodes; they can be updated and queried in real time or on-demand. Hence, the major challenge is how to efficiently handle the large amount of objects and data, all in a global information space, with a performance that can meet real-time requirements.

Big data is closely related to *cloud computing* [2]. A cloud environment consists of hardware and software resources placed remotely and accessed over a network. The physical infrastructure is virtualized and abstracted to users, providing a virtually unlimited resource capacity on-demand

as well as easy administration. Cloud databases [3] are an important part of the cloud infrastructure. To deal with huge volumes of data, such databases use the cloud computing paradigm to optimize scalability, availability, multitenancy, and resource usage.

To this end, many new technologies have been created to handle the data growth while improving the performance. Different database management systems (DBMS) have been developed and evaluated to this end. Even though SQL databases have been the classic and dominant type among database systems so far, questions have been raised whether traditional relational databases fit well the new data types and performance requirements of recent application scenarios [4, 5]. In this context, NoSQL databases have emerged as a response to such changing demands. They store data very differently from traditional relational database systems. NoSQL databases are schema-free, i.e., they are designed for data that could be unstructured. They are claimed to be easily distributed with high scalability and availability. These properties are indeed needed to realize the vision behind the IoT from the data perspective.

Regarding IoT data, an open question is how to manage storage systems in an efficient and cost-effective way. This depends on a proper planning on which DBMS is used to store the data and how the database is configured to provide adequate performance. Although a variety of databases is currently available, which model and solution best fits IoT data is still an open problem. To the best of our knowledge, there has not been much research on a general storage solution for IoT data that provides a practical, experimentally-driven characterization of the efficiency and suitability of different databases, especially in the cloud environment.

This article specifically addresses such a scenario and targets a solution that can provide the best performance for the different data types and the massive amount of IoT data. To this end, we investigate different types of cloud databases; we evaluate and compare NoSQL databases against traditional SQL databases, by pointing out their differences in performance, usage and complexity. Our focus is to evaluate the most common types of IoT data, namely, sensor readings and multimedia. Extensive experiments have been performed with four popular databases: MySQL, MongoDB, CouchDB and Redis, with focus on MongoDB versus MySQL. All the database servers considered were running in the cloud.

The rest of the article is organized as follows. Section II

overviews the related work on cloud databases for IoT data. Section III briefly introduces the databases used, then details the methodology and the setup of the experiments performed. Section IV presents and discusses the experimental results. Finally, Section V summarizes and concludes the article, with directions for future research.

II. RELATED WORK

A. Performance of SQL and NoSQL Databases

A large number of studies have addressed database performance, even though many of them are limited to a small scale or case-specific local experiments. Several studies have also been carried out to characterize the performance of different databases under real workloads.

Datastax Corporation examined three NoSQL databases: the Apache Cassandra key-value store, the column-oriented Apache HBase, and the MongoDB document store, all running on Amazon EC2 m1 extra large instances [6]. The results showed that Cassandra outperformed the other two solutions by a large margin; MongoDB exhibited the worst performance. However, there was no SQL database involved, nor was there any other document database to compare with MongoDB, which is instead the main focus of our evaluation.

In [7], the elasticity of NoSQL databases – including HBase, Cassandra, and Riak – was evaluated and compared in terms of the changes in the query throughput when the server cluster size changed. The results showed that HBase was the fastest and the most scalable solution when the system was read-intensive. On the other hand, Cassandra performed and scaled well in a write-intensive environment, where nodes could be added without a transitional delay time. Furthermore, the authors proposed a prototype of a module able to automatically resize a cluster in order to meet target performance requirements.

The work in [8] addressed storing application performance management data and analyzed the scalability and performance of six databases, i.e., MySQL and five NoSQL variants. The benchmark showed the latency and throughput of the considered databases under different workloads. Again, Cassandra was the clear winner in the experiments, while HBase obtained the lowest throughput. When it comes to sharding, which means partitioning data horizontally across multiple servers, MySQL achieved nearly as high throughput as Cassandra. Although a standalone Redis solution outperformed the others when the system was read-intensive, its performance in a sharded implementation dropped as the number of nodes increased. VoltDB exhibited a similar behavior in a sharded system, thus not scaling very well.

MySQL, Cassandra, HBase and Sherpa were compared in [9]. The experiments concluded that MySQL was not as efficient as the NoSQL databases for storing a massive volume of data, especially in write-intensive scenarios. How-

ever, MySQL showed a relatively high performance in read-intensive scenarios.

All the works mentioned above have one common aspect, i.e., they employed the Yahoo! Cloud Serving Benchmark (YCSB), a generic evaluation framework suitable for key-value stores. YCSB helped to perform experiments with a large amount of data: hundreds of millions of records, hundreds of clients, multiple data nodes, and different read and write intensities. However, little attention was paid on the structure and variety of data.

B. Internet-of-Things Storage

With specific reference to IoT data, PostgreSQL, Cassandra, and MongoDB were evaluated in [10] as a storage solution for sensor readings. Instead of a real cloud environment, the experiments were run between a physical server and a virtual machine. The work is closely related to ours for the similar data structures used. The results did not show a single winner as MongoDB performed well for single writes and PostgreSQL for multiple reads. Furthermore, the impact of virtualization was unclear.

Several solutions for storing IoT data have also been proposed. A storage system for massive IoT data based on the NoSQL approach, IOTMDB, was discussed in [11]. The proposed solution includes strategies for expressing common IoT data in the form of key-value pairs, as well as a data preprocessing and sharing mechanism.

Paraimpu was introduced in [12] as a social web-based platform for IoT data. The proposed system allows to build a Web of Things connecting HTTP-enabled smart devices such as sensors and actuators with virtual “things” such as services, social networks, and APIs. The platform adopts MongoDB as database server, provides models and interfaces that help to abstract and adopt different types of data and devices.

SeaCloudDM was proposed in [13] as a cloud data management framework for sensor data. The proposed solution addressed the challenges related to data that are dynamic, various, massive, and spatial-temporal (i.e., each sample corresponds to a specific time and location). To provide a uniform storage mechanism for heterogeneous sensor data, the system combined the use of the relational model and the key-value model, and was implemented with a PostgreSQL database. Its multi-layer architecture was claimed to reduce the amount of data to be processed at the cloud management layer. Besides, the work also provided several experiments that showed a promising performance when storing and querying a huge volume of data.

A document-oriented data model and storage infrastructure for heterogeneous and multimedia data in the IoT was presented in [14]. The system employed CouchDB as database server, taking advantage of its RESTful API and other features such as replication, batch processing, and change notifications. The authors also provided an

optimized document uploading scheme for multimedia data that showed a clear performance improvement.

In this article, we target data types similar to those in [14] but we rather focus on the read and write performance. We provide more extensive experiments and compare the performance of different types of database (both SQL and NoSQL, including document and key-value stores). Besides, we evaluate the considered databases in a real cloud environment.

III. EXPERIMENTAL METHODOLOGY AND SETUP

The main goal of the experiments was to compare the performance of different solutions as cloud databases with typical IoT data. Hence, the database servers were placed on an Amazon EC2 instance in the cloud.

The experiments were run on four open-source databases: MySQL [15], MongoDB [16], CouchDB [17], and Redis [18]. We chose the databases among those that were both popular and representatives for their type (e.g., both SQL and NoSQL).

The evaluation targeted two particular data types in two separate benchmarks: *scalar sensor data* and *multimedia data*. With the variety of Internet-of-Things data, the data types above are expected to cover many different scenarios, in terms of amount, data formats, and applications.

In the benchmarks, the database clients were implemented to perform basic read/write operations. In the experiments, we run these operations with different workloads, measured the average request latency, and compared it across the different databases considered. Each operation was assessed separately, i.e., only one type of database was evaluated at any time, only one experiment was running, and the system was under 100% read load or 100% write load. Each experiment was a single or a continuous series of either read or write requests from clients to a database. The different experiments were setup by using different parameters, e.g., the number of records or the number of concurrent clients (simulated by multiple threads).

The performance measurement was done at the client side. Specifically, the time taken to complete the requests was measured in each experiment; the times for connecting to the database and for the actual execution of the requests were recorded separately. One limitation of the setup was that the network connection between clients and servers was not dedicated and, thus, could not be controlled. Hence, in order to increase reliability, each experiment was run multiple times (at least 10 times) with the same input. The final result for an experiment was the average value over such individual runs.

The benchmarks were implemented in Java and ran on the Java 7 Virtual Machine (JVM). The details on the hardware and software platforms as well as of the database servers and clients are given in Table I. All databases ran with the default

TABLE I. EXPERIMENT HARDWARE AND SOFTWARE PLATFORMS

Parameter	Server	Client
Machine	Amazon EC2 m1.large, Europe West datacenter	7.5 GiB RAM, Intel Core2 Quad Q6600 at 2.4 GHz
OS	64-bit Ubuntu 12.10	64-bit Ubuntu 12.04
MySQL	5.6.10	Connector 5.1.22
MongoDB	2.4.3	Java Driver 2.10.1
CouchDB	1.2.0	Ektorp 1.2.2
Redis	2.6.12	Jedis 2.1.0

configurations, except for the *data* and *log* file paths that were set to a storage volume of the Amazon EC2 instance.

A. Scalar Sensor Data Benchmark

This benchmark was used to evaluate the efficiency of *MySQL*, *MongoDB*, *CouchDB*, and *Redis* when storing scalar readings generated by sensor nodes. The benchmark was built based on the Home Energy Management System developed by There corporation [19].

The implemented system simulated a network consisting of a central *database server* (located in the cloud) and multiple uniquely-identified *sensor nodes*. Each node generated a reading (i.e., a record) once in an interval. In the implementation, to simulate the multiple sensor readings, we used a *data generator* that created a list of sensor records with random values which would then be inserted into the server by a *data sender*. Since data were sent continuously, we assumed that performance and availability had higher priority than data integrity in such a system.

The databases adopted the two different designs detailed below. MongoDB was tested with both designs in order to evaluate the impact of the data structures on the system performance.

Single data set: MySQL, MongoDB 1set, CouchDB
Records of all nodes were stored in only one common set, thus making it possible to use *bulk insert* and improve the write performance, as there was only one destination storage.

Multiple data sets: MongoDB mset, Redis
In this case, one database consisted of multiple collections (multiple *hashes* in case of Redis), each dedicated to one node; the collection name was the *nodeID*. Data sent to the database were distributed to the corresponding collections. This design reduced the duplication of the *nodeID* field in every record. Besides, querying for data of a single node, which we considered the most popular query, was simpler and only involved a small set of data rather than all of them.

B. Multimedia Data Benchmark

The purpose of this benchmark is to evaluate the performance of SQL and NoSQL databases when used for multimedia storage on the cloud. Multimedia data can be of different formats and from any kind of application. For instance, they could be pictures on social networks, audio on music streaming sites, or video from surveillance cameras. In this benchmark, *MySQL* and *MongoDB* were chosen

TABLE II. SETTINGS FOR THE BULK INSERT EXPERIMENTS

Parameter	Value
Total number of records	10,000
Number of sensor nodes	100
Number of records generated by each sensor node	100
Number of concurrent writing threads	1
MongoDB index for NodeID	True

as representative databases. The removal of CouchDB and Redis was due to their limited performance for the sensor data benchmark when compared to MongoDB, in terms of both latency and storage capacity. Further details will be given in Section IV.

The reference scenario is represented by *media senders* that continuously send multimedia files to a server, one at a time. Each file was identified by a unique filename, which could be, for instance, an auto-incremented counter or the time the file was created. The database clients could then query for the content of a file by its filename. In the experiments, all files had the same size and format.

To store multimedia files of large size, MySQL uses the *blob* (Binary Large Object) data type. Meanwhile, we used the *GridFS* feature for MongoDB [16], where the database consisted of two collections: the *files* collection storing the file metadata, and the *chunks* collection storing the actual binary data, divided into small pieces. Each file was given a unique *filename*, i.e., a counter incremented by one for each file inserted.

IV. EXPERIMENTAL RESULTS

A. Scalar Sensor Data Benchmark

1) *Bulk insert vs individual insert performance*: Instead of adding data once at a time, clients can use a feature called *bulk insert*, through which they insert multiple records in a single operation. Bulk insert is expected to improve write performance; hence, the purpose of this experiment is to evaluate the efficiency of the bulk insert over individual inserts on the group of databases with a single data set, i.e., MySQL, MongoDB 1set and CouchDB. In fact, bulk insert only works on a single table (or collection) and is not applicable for the other group of databases, i.e., MongoDB mset and Redis. We used the parameters shown in Table II for all the experiments.

Figure 1a shows the results for the bulk insert experiments. Clearly, bulk insert has a significant impact on the performance of the three databases, as individual inserts cause a much higher latency. Among the three databases, MongoDB had the best results in general. The insert time decreased gradually along with the increase in bulk size. A similar pattern applied to CouchDB, which was anyway outperformed by MongoDB in all cases. In contrast, MySQL obtained a latency that decreased until reaching the minimum at the bulk size of 400 records. After this point, the latency significantly increased. It is worth noting

TABLE III. SETTINGS FOR THE WRITE LATENCY EXPERIMENTS

Parameter	Value
Number of sensor nodes per thread	1,000
Number of records generated by each sensor node	100
MongoDB index for NodeID	True
MySQL, MongoDB 1set, CouchDB bulk insert size	1000
MongoDB mset, Redis bulk insert size	1

TABLE IV. SETTINGS FOR THE READ LATENCY EXPERIMENTS

Parameter	Value
Query type	Fetch all data of one sensor node
Number of sensor nodes	1,000
MongoDB index for NodeID	True

that an individual insert in MySQL was implemented by using *prepared statements*; as a consequence, the overhead of compiling and optimizing similar queries occurred only once. Thus, the performance was vastly improved compared to using normal statements.

Apart from affecting the write latency, bulk inserts also caused a major reduction in the database size for CouchDB, although it was not the case for the other databases. This was due to the use of *append-only Btree* data structure employed by CouchDB to store documents. The storage used for individual inserts was of 33.5 MiB compared to the 2.5 MiB when using bulk inserts of 100 elements at once, i.e., about 14 times higher. Although the data size reduced as the bulk size increased, this difference was not significant.

2) *Multi-User Write Performance*: The write performance was assessed in a multi-threaded environment (not only multiple *sensor nodes* but multiple *data senders*), therefore different numbers of threads were employed for different experiments. One thread corresponded to a *data sender*, each in charge of 1,000 *sensor nodes*. As a consequence, each time a group of sensors generated samples, 1,000 records were sent to the database all at once. The number of such generation rounds was set 100. As a result, each thread inserted a total of 100,000 records into the database. According to the previously discussed results, we used bulk inserts for MySQL, MongoDB 1set and CouchDB, with the same bulk size as the number of sensor nodes. Thus, all 1,000 records were inserted together in a single operation. Since bulk insert was not applicable for MongoDB mset and Redis, we used individual inserts instead, i.e., the 1,000 records were added as 1,000 successive operations. Table III summarizes the parameters used in the experiments.

Figure 1b illustrates the average values of write latency with respect to different numbers of concurrently writing threads. The values exclude the connection time, since we expected the sensor nodes to send data continuously without disconnecting. The figure shows an obvious difference between the *multiple data set* group that used individual insert (MongoDB mset and Redis) with higher latency and the *single data set* group that used bulk inserts. In this benchmark, the choice for the bulk size was made as a

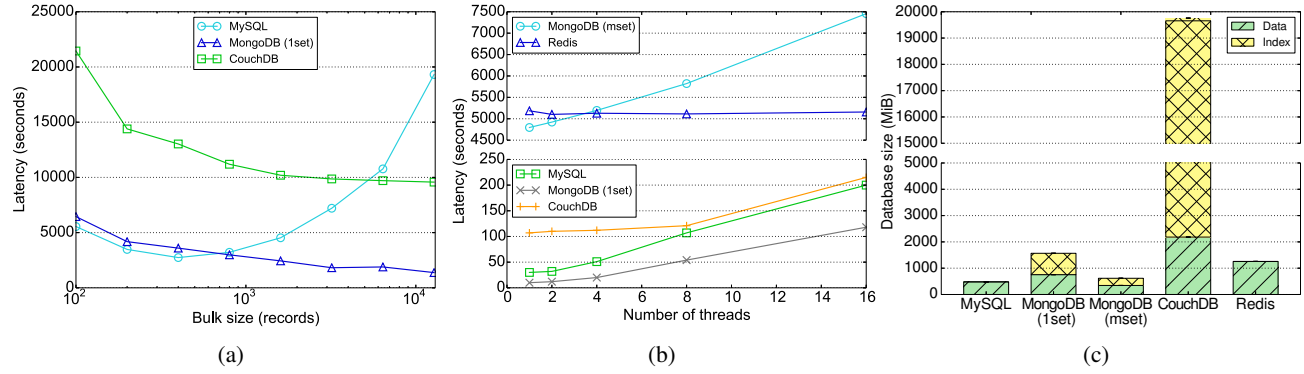


Figure 1. Latency for (a) bulk insert and (b) multi-user write. (c) Database size for the different solutions considered.

compromise reflecting how the system could be used in real life, while providing a general comparable view among the databases. In other words, the results do not mean that Redis' write performance is the slowest among the different databases; they rather show the disadvantages in using Redis for this particular type of system and data. In the group of databases with one data set, MongoDB 1set obtained the best results, followed by MySQL and CouchDB. Furthermore, the results for MongoDB mset and Redis were more or less the same with a small number of threads, even though the former became slower than the latter as the number of threads increased.

Regarding concurrency, Redis was hardly affected by the number of threads as all operations were performed in RAM. However, the trend was different for the other databases as the latency increased with the number of threads.

3) *Database Size*: Although storage space is relatively inexpensive, as the capacity of hard disks is increasing dramatically while the price is decreasing, it is still an aspect that is worth consideration. To this end, Figure 1c illustrates the size of a database containing 10 million records for the different solutions considered.

The figure shows how CouchDB occupies much more space than the rest of the databases, i.e., around 40 times more than that of MySQL (the one with the smallest size). A large portion of such a huge size came from the view indexes, representing nearly 90% of the total. Moreover, there was only one index (built for the query of fetching data of each node) in this experiment; the size would have been higher if more views had been created. Furthermore, there was no extra index in Redis or MySQL. This was because Redis did not support a secondary index. For MySQL, there was no need to build a secondary index in addition to the automatically indexed primary key that physically sorts the records. Since Redis is an in-memory database, it has a strong disadvantage compared to the other ones, as the storage capacity is limited by the amount of RAM. Besides, the database size for Redis was larger than that of MySQL and MongoDB mset, which had the same design of multiple

data sets.

For MongoDB, the design of multiple data sets improved the storage usage compared to the single set, since it avoided the duplication of the *nodeID* field and promoted the *time* as the unique *_id* instead of using a system-generated *ObjectId*. This made MongoDB mset quite efficient in using the storage space, only slightly larger than MySQL. Besides, for document stores like CouchDB or MongoDB, using shorter field names (which are repeated in every document) can help save space.

4) *Multi-User Read Performance*: The aim of the following experiments is to assess the query performance of the databases under different loads, including different data volumes and different number of concurrent queries. For this scenario, we assumed that the most popular query was the one to get all the data of a particular sensor node and used that query for all the experiments. Finally, each thread queried for a different node and all nodes had the same amount of data.

Figure 2 shows the average query latency (including the connection time) with respect to different numbers of threads in several experiments, each with a different database size. It is apparent how CouchDB was significantly outperformed by all the other databases, while the differences among the rest of them were less remarkable. In general, Redis got the best results due to its in-memory storage and the fact that data are queried by key in this scenario. In the cases where the database size was large (10,000,000 records or more), MySQL was close to Redis and also had a short latency. Querying by the primary key index in MySQL, as in this scenario, achieves optimal performance since the data are physically sorted by the primary key. Moreover, between the two types of MongoDB designs, the change in data structure from one collection (MongoDB 1set) to multiple collections (MongoDB mset) improved the performance of the query in all cases as expected. Finally, in some cases with a single thread, MongoDB mset was slightly faster than the others.

It is worth noting that all these queries were run after the databases had been "warmed up", which means the queries had been run once before the experiments started. This was

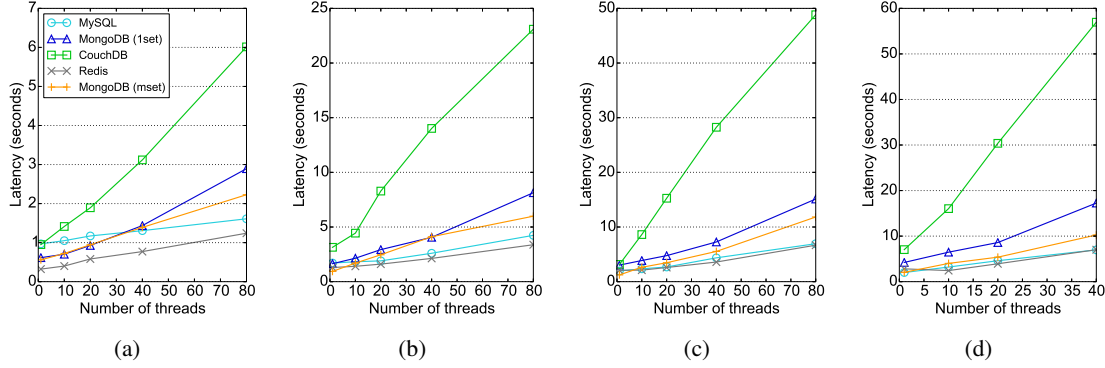


Figure 2. Query latency as a function of the number of threads for different number of records: (a) 1 M, (b) 10 M, (c) 20 M and (d) 40 M.

to make the data ready to be queried in the fastest mode. The warm-up did not have an impact on Redis, since all the data had already been in RAM. Nevertheless, it had a significant impact on CouchDB, since the view index was created on the first run, which indeed took long time. For the other databases, the first run loaded the data set of MongoDB into memory and the query results of MySQL into the query cache, which highly boosted the performance afterwards. However, the impact of this warm-up time would not be so pronounced in a real scenario as in the experiments, since data may be changed and added continuously, or the amount of data may not fit in the cache. For the latter case, a horizontal scaling solution should be considered.

B. Multimedia Data Benchmark

1) *Multi-User Write Performance*: The experiments below were run to compare the write performance of MySQL and MongoDB when storing multimedia files. Since we assumed that all the files had the same size and format, the two parameters that could affect the results were: the size of each data item; and the number of concurrent inserting threads. In detail, the write latency was measured as the time taken to insert 1,000 MP4 video files per thread. We excluded the connection time, as we assumed that the system generated data continuously.

The results of the experiments are illustrated by Figure 3. The figure shows a clear difference between MySQL and MongoDB, where the former outperformed the latter. It is also apparent how the performance of both databases highly depended on the file size and the number of threads, as the latency increased proportionally with the increase of the item size and the number of threads. The figure also shows that MongoDB was close to MySQL when the server handled more clients at the same time. While MySQL stored the files as single blobs, MongoDB's GridFS divided and stored them as small chunks along with the files metadata, which added extra information and some more latency to the operation. However, the latter approach is useful for clients querying for a portion of a certain file. GridFS is also beneficial for systems requiring horizontal scaling as chunks of huge files

can be shared onto multiple servers.

2) *Multi-User Read Performance*: Among the parameters that could be set, the query performance is affected by three major parameters: the size of each data item, the total number of items in the database; and the number of concurrent querying threads.

The purpose of the following experiments is, therefore, to find out the impact of these parameters on the latency of querying for a random file in the database. The latency recorded in the experiments includes both the connecting and querying times. The querying time itself consists of the time to scan through the database to find the right record (for MongoDB it means searching through both the *files* and *chunks* collections) plus the time to read the binary data from the database and write it to a local file at the client machine.

Figures 4a–4c illustrate the impact of file size on the read latency. In all cases, the data set contained 1,000 MP4 video files of the same size. The figures do not show a significant difference between MySQL and MongoDB in this scenario; however, they do show a clear impact of the data item size on the read performance, since the time taken for reading one item roughly doubled as the file size doubled.

Figures 4d–4f show the impact of the total number of records on the time taken to search for one file in the database. Now that the queried data are of the same size, the difference in the latency is purely due to the time taken to locate and read the data. However, the graph does not show a clear relation between the query time and the total number of records, as for both MongoDB and MySQL there are fluctuations. Anyway, we believe that the total database size does not have a significant impact on querying for a file, as the time taken to locate the file is very small compared to the actual time of reading the binary data and writing it to a local file.

In general, the results in Figure 4 share several common aspects. First, the performance dropped when there were more threads querying at the same time. Second, the difference between MySQL and MongoDB was inconsistent and small; it was less than 1 second in most cases. Although

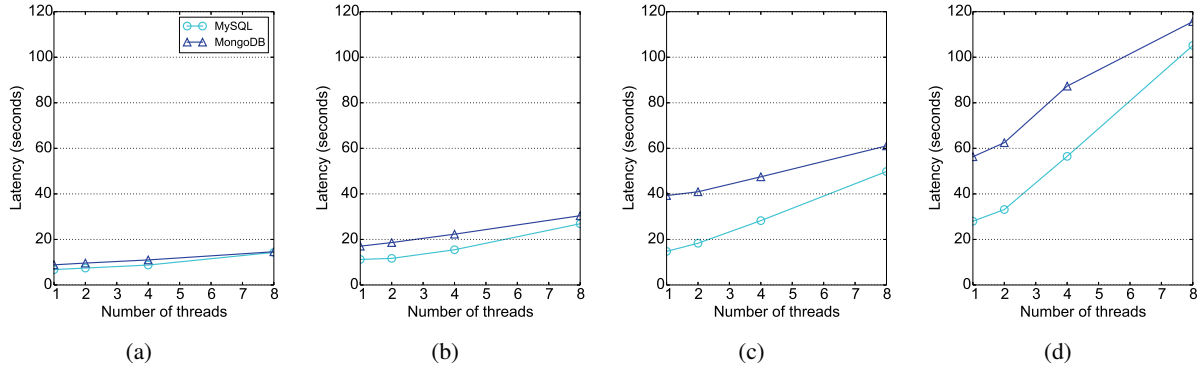


Figure 3. Media write latency as a function of the number of threads for different file sizes: (a) 1 MiB, (b) 2 MiB, (c), 4 MiB and (d) 8 MiB.

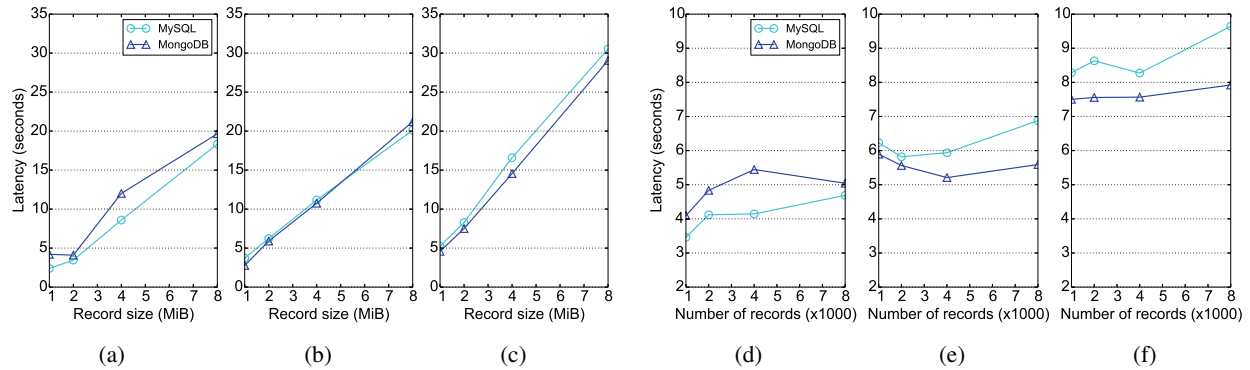


Figure 4. Media query latency as a function of the record size for different number of threads: (a) 1, (b) 20, and (c) 40. Media query latency as a function of the number of records for different number of threads: (d) 1, (e) 20, and (f) 40.

it may seem that MongoDB performed slightly better when there were more threads, it is hard to compare and conclude about the query performance of the two databases.

V. DISCUSSION AND CONCLUSION

The purpose of this article was to investigate how different database systems can effectively handle a heterogeneous and large amount of Internet-of-Things data in the cloud, in order to meet the increasing demands on both load and performance. Two classes of databases were studied, namely, SQL and NoSQL databases. Several benchmarks were conducted on four different solutions: MySQL, MongoDB, CouchDB, and Redis. The benchmarks compared the read and write performance of the databases as a storage for two popular types of IoT data: scalar sensor data and multimedia data.

The scalar sensor data benchmark showed good results for the NoSQL databases in write-intensive systems with the use of bulk inserts, especially MongoDB followed by MySQL, CouchDB, and Redis. In contrast, the difference in the performance obtained for querying data was less pronounced. Redis managed to achieve the best results in general, and MySQL performed nearly as fast in most cases. Although MongoDB lagged behind, the difference was acceptable, especially considering that the reference

scenario was write-intensive and MongoDB outperformed the rest of the solutions when executing data insertions.

In contrast, the performance of CouchDB was rather poor in our experiments, not to mention its huge database size compared to the others. Redis also had similar issues: such a key-value in-memory database is limited by the database size, data structure, and query capabilities. Hence, Redis and CouchDB do not appear to be the best choices for a system serving big IoT data and real-time queries.

In our experiments, MongoDB was employed with two different designs. The single collection design is more suitable for the considered scenario than the one with multiple collections. That is because switching from the former to the latter may result in a slight improvement for querying data but causes a huge loss in the write performance. The lesson learned is to take advantage of the flexible schemaless data model and consider the best fit for the system, since a change in the data model can make a huge change in the performance.

Following the results of the scalar sensor data benchmark, we conducted a similar study with multimedia data on MySQL and MongoDB. The results showed that MySQL using blob storage performs better than MongoDB's GridFS when it comes to inserting multimedia files. As for the query

performance, the difference between the two databases was unclear, even though MongoDB was slightly faster when serving more clients simultaneously. However, since multimedia data tend to be large, the approach of MongoDB's GridFS makes it easier to shard the database across several machines, thus distributing the loads and increasing the scalability.

In conclusion, it is hard to point out the best cloud database for the IoT, since the data types are varying and the scope of the relevant use cases is vast. Moreover, each database has its own pros and cons, as well as its own area of application. Which database to choose, therefore, highly depends on the properties and the requirements of the specific system. However, for the scenarios that were studied here, our results have shown the potential of NoSQL databases against traditional relational database systems.

There is still room for future research about this problem. One is to expand the current benchmarks to further explore the performance of the databases with more complex types of IoT data, for instance, an object-oriented data model involving multiple object types. A similar direction is to investigate the strength of the schema-free data model against the powerful (yet expensive) joining of data across multiple SQL tables. Another option is to assess the efficiency of scaling the system by sharding and replication under failures. Scalability is actually one key point that can potentially make NoSQL preferable over SQL databases, by considering the fact that most NoSQL databases were originally designed to scale out seamlessly to meet the growing demand of Internet data.

ACKNOWLEDGMENTS

This work was partially supported by TEKES as part of the Internet of Things program of DIGILE (the Finnish Strategic Center for Science, Technology and Innovation in the field of ICT and digital business) and by the Flexible Spaces Services activity of the EIT ICT labs.

REFERENCES

- [1] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [2] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica *et al.*, "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [3] V. Mateljan, D. Ciscic, and D. Ogrizovic, "Cloud database-as-a-service (DaaS) – ROI," in *MIPRO, 2010 Proceedings of the 33rd International Convention*. IEEE, 2010, pp. 1185–1188.
- [4] C. Strauch, U.-L. S. Sites, and W. Kriha, "NoSQL databases," *Lecture Notes, Stuttgart Media University*, 2011.
- [5] E. Lai, "No to SQL? anti-database movement gains steam," *Computerworld Software*, July, vol. 1, 2009.
- [6] Datastax Corporation, *Benchmarking Top NoSQL Databases*. Datastax, 2013.
- [7] I. Konstantinou, E. Angelou, C. Boumpouka, D. Tsoumakos, and N. Koziris, "On the elasticity of NoSQL databases over cloud management platforms," in *Proceedings of the 20th ACM international conference on Information and knowledge management*. ACM, 2011, pp. 2385–2388.
- [8] T. Rabl, S. Gómez-Villamor, M. Sadoghi, V. Muntés-Mulero, H.-A. Jacobsen, and S. Mankovskii, "Solving big data challenges for enterprise application performance management," *Proceedings of the VLDB Endowment*, vol. 5, no. 12, pp. 1724–1735, 2012.
- [9] B. G. Tudorica and C. Bucur, "A comparison between several NoSQL databases with comments and notes," in *Roedunet International Conference (RoEduNet), 2011 10th*. IEEE, 2011, pp. 1–5.
- [10] J. S. van der Veen, B. van der Waaij, and R. J. Meijer, "Sensor data storage performance: SQL or NoSQL, physical or virtual," in *Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on*. IEEE, 2012, pp. 431–438.
- [11] T. Li, Y. Liu, Y. Tian, S. Shen, and W. Mao, "A storage solution for massive IoT data based on NoSQL," in *Green Computing and Communications (GreenCom), 2012 IEEE International Conference on*. IEEE, 2012, pp. 50–57.
- [12] A. Pintus, D. Carboni, and A. Piras, "Paraimpu: a platform for a social web of things," in *Proceedings of the 21st international conference companion on World Wide Web*. ACM, 2012, pp. 401–404.
- [13] Z. Ding, J. Xu, and Q. Yang, "SeaCloudDM: a database cluster framework for managing and querying massive heterogeneous sensor sampling data," *The Journal of Supercomputing*, pp. 1–25, 2012.
- [14] M. Di Francesco, M. Raj, N. Li, and S. K. Das, "A storage infrastructure for heterogeneous and multimedia data in the Internet of Things," in *The 2012 IEEE International Conference on Internet of Things (iThings 2012)*, November 2012, pp. 26–33.
- [15] Oracle Corporation, "MySQL documentation: MySQL 5.6 reference manual," <http://dev.mysql.com/doc/refman/5.6/en/>, Accessed: 16.05.2013.
- [16] "The mongodb manual," <http://docs.mongodb.org/manual>, Accessed: 16.05.2013.
- [17] J. C. Anderson, J. Lehnardt, and N. Slater, *CouchDB: The Definitive Guide: Time to Relax*. O'Reilly Media, 2010.
- [18] K. Seguin, "The little Redis book," *Karl Seguin*, 2010.
- [19] "There corporation," <http://www.therecorporation.com/en/products/>, Accessed: 04.06.2013.